

600089 A25 T2: Distributed Systems Programming Report

May 2026

By

Ionut-Adrian Velicu

Student number 202342668

Word count: 1499

Intro

The project is to create a distributed system and be able to handle the client side and the server side. In order to do this the program uses ASP.Net Core Web API as the backbone with added authentication and cryptography via services such as TalkBack, UserController, ProtectedController and the pattern was used for the routing is `api/{controller}/{action}`.

The way this was all managed was through the use of requests. Specifically the server would handle multiple different instances of the client requests with user controllers being used to route these requests and lastly there's middleware used for authenticating the user.

More specifically the CustomAuthenticationHandlerMiddleware purpose is to check if the API key is valid and see if an existing user matches the same key or not. As for the authorization the CustomAuthorizationHandlerMiddleware checks what the user is allowed to do once they're in the system e.g., if they have admin privileges or not and blocks them from accessing sensitive endpoints.

Additionally, the program architecture is stateless, this means that each request must contain all the data required to process that request e.g., requiring the APIKey of the user. Meaning that the Server doesn't store session information between user requests. Furthermore it allows for multiple simultaneous requests without any shared states between threads which is a part of concurrent client handling.

However, if the server were stateful then it would store that user data between requests. It does mean that it's more convenient for the user since they don't have to send their APIKey

each time someone makes a request. The downside is that it would make scaling harder since every request would need to reach the same server that holds that session.

Systems Architecture

Since the project was built as a client-server architecture. There are two separate applications that were used to achieve this effect. The web server which uses [ASP.NET](#) core Web API and the client side app programmed in C#. The glue that helped to communicate between these two systems is done through HTTP requests.

The responsibilities of the client application are as follows: Track user commands, Process inputs, and send those requests to the server by using the HttpClient class. Furthermore, important information such as the user's API key and the server public RSA key are stored client side during runtime. For example, commands such as TalkBack Hello, Protected SHA256 hello, and Protected Mashify Hello World! All are processed through the client and then converted to API requests.

Then for the server side it's main responsibilities are handling the different API controllers such as TalkBackController, UserController, ProtectedController.

The TalkBackController manages the basic communication and sorting functionality, the UserController manages the user role creation, API key generation and deleting users. Finally the ProtectedController is what contains the endpoints for authentication and authorization, this includes hashing, RSA signing, AES encryption and Mashify functionality.

Authentication and authorization are done through custom made middleware components. How this works is that CustomAuthenticationHandlerMiddleware validates user API keys and identify any users on the database, while the CustomAuthorizationHandlerMiddleware checks for what role the user currently has and makes sure certain key areas can only be accessed with the appropriate admin privileges.

The project also utilizes a database system through Entity Framework Core. The database stores users, API keys, and request logs. A table is generated based on the logs, said logs can then check what requests were made to protected endpoints with timestamps and API info providing a full audit trail of activity. Additionally, route mapping was implemented using this pattern `api/{controller}/{action}` which was called in BaseController, where all the controllers inherit from. It allows to automatically map URLs to controller methods e.g., `/api/user/new` then routes to the new action in UserController.

Alternatively, this could have been approached to call each method and route it there e.g., `[Route("api/user/new")]`, the benefits are that there's more fine-grain control but it's a bit more complex to configure per each endpoint.

Figure 1 below showcases the architecture and flow of communication through several components such as the client, middleware, API controllers and database systems.

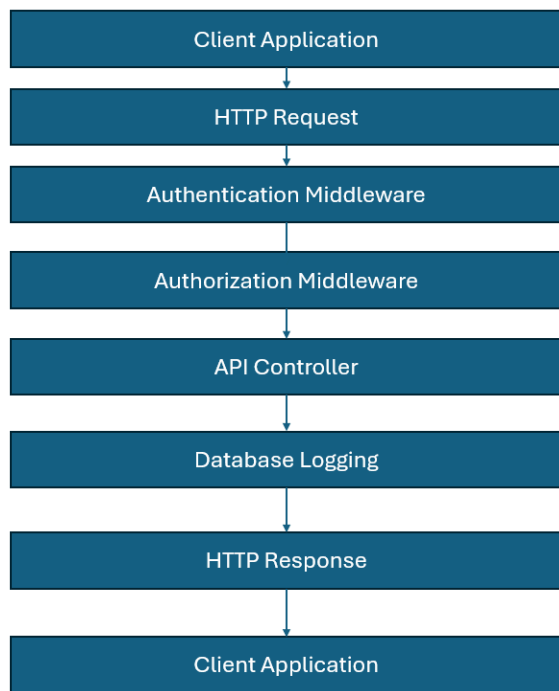


Figure 1: Systems architecture and flow diagram

Authentication and Authorization

Authentication and authorization were implemented as middleware components through ASP.NET Core Web API server. These systems were used to improve security and make sure valid users with the correct role can access sensitive areas such as protected endpoints.

Authentication is handled through CustomAuthenticationHandlerMiddleware. It checks the incoming HTTP requests and checks to see if the API key stored matches what the database has through the use of Entity Framework Core. If it is valid then the user would be authenticated and can access the rest of features there. If the API was invalid or missing then the user cannot access the rest of the protected endpoints of the server.

Once validated, the middleware builds a ClaimsPrincipal which contains two types of claims one being ClaimTypes.Name for storing usernames and ClaimTypes.Role for storing the user's role. These are then packaged into an AuthenticationTicket allowing the [Authorize] attribute on each controller to function correctly.

As for Authorization this was handled separately through CustomAuthorizationHandlerMiddleware. This works after a user is successfully authenticated it checks what the user's roles are and what they can access. For example, regular users can access most protected endpoints such as Protected Hello, SHA1 and SHA256, however Admin-only endpoints such as User ChangeRole and Protected Mashify are restricted using [Authorize(Roles = "Admin")]. Any non-admin attempting to access these

receives a 403 Forbidden response with the message 'Forbidden. Admin access only.' The only way for a user to do that is to change the role to be an admin instead ensuring security with this role-based access control in the application.

Although API keys are easy to implement, they can pose security concerns. Keys are long-lived and if they are intercepted can give full access to the system unless manually revoked. As such in a production system utilizing JWT tokens that expire much faster and require refreshing would be more secure. As well as transmitting all requests over HTTPS to encrypt the key in transit, and storing keys as hashed values in the database rather than plaintext.

Lastly, security functionality such as SHA1, hashing SHA256, RSA digital signing, AES encryption, Mashify encryption/decryption were all implemented inside of ProtectedController. This further adds more security to the program and makes sure the data sent to and from is secured.

```
[ApiController]
[Route("api/[controller]")]
1 reference
public class UserController : BaseController
{
    private readonly UserDatabaseAccess _dbAccess;

    0 references
    public UserController(UserContext dbContext, SharedError error) : base(dbContext, error)
    {
        _dbAccess = new UserDatabaseAccess(dbContext);
    }
    //changed it to new? will it work
    [HttpGet("New")]
    0 references
    public IActionResult GetNew([FromQuery] string? username)
    {
        if (!string.IsNullOrEmpty(username) && _dbAccess.UserNameExists(username))
        {
            return Ok("True - User Does Exist! Did you mean to do a POST to create a new user?");
        }

        return Ok("False - User Does Not Exist! Did you mean to do a POST to create a new user?");
    }
}
```

Figure 2: GET Code Snippet

```

//checking user roles and if they are valid (error checking) or not (TASK 8)
[HttpPost("ChangeRole")]
[Authorize(Roles = "Admin")]
0 references
public IActionResult ChangeRole([FromBody] ChangeRoleRequest? request)
{
    try
    {
        if (request == null || string.IsNullOrEmpty(request.Username))
        {
            return BadRequest("NOT DONE: Username does not exist");
        }
        if (!_dbAccess.UserNameExists(request.Username))
        {
            return BadRequest("NOT DONE: Username does not exist");
        }

        if (request.Role != "User" && request.Role != "Admin")
        {
            return BadRequest("NOT DONE: Role does not exist");
        }

        bool changed = _dbAccess.ChangeUserRole(request.Username, request.Role);
        if (changed)
        {
            return Ok("DONE");
        }
        return BadRequest("NOT DONE: An error occurred");
    }
    catch
    {
        return BadRequest("NOT DONE: An error occurred");
    }
}

```

Figure 3: POST Code Snippet

```

//removes user (Task 7 DONE)
[HttpDelete("RemoveUser")]
[Authorize(Roles = "Admin, User")]
0 references
public IActionResult RemoveUser([FromQuery] string? username)
{
    if (string.IsNullOrEmpty(username))
    {
        return Ok(false);
    }

    string? apiKey = Request.Headers["ApiKey"].FirstOrDefault();

    if (string.IsNullOrEmpty(apiKey))
    {
        return Ok(false);
    }

    User? requester = _dbAccess.GetUserByApiKey(apiKey);

    if (requester == null)
    {
        return Ok(false);
    }

    bool isDeletingSelf = requester.UserName == username;
    bool isAdmin = requester.Role == "Admin";

    if (!isDeletingSelf && !isAdmin)
    {
        return Ok(false);
    }

    bool deleted = _dbAccess.DeleteUserByUserName(username);
    return Ok(deleted);
}

```

Figure 4: DELETE Code Snippet

Cryptography Implementation

The protected controller utilizes multiple ways to combat security.

SHA1 and SHA256 for hashing user messages. How this works is that these algorithms are one-way only making the original message hard to reverse engineer from the new hash. SHA256 is a more powerful version offering more security at the expense of larger data size compared to SHA1.

RSA cryptography was also used for digital signing functionality. The server generates both a public and private key.

The private key remains server-side and is used to digitally sign messages and decrypt incoming data, while the public key is shared with the client for encryption and signature verification. Utilizing both like this ensures the message authenticity is there while making sure the message wasn't modified during requests.

Additionally, RSA operates by using OAEP SHA1 padding mode which ensures compatibility and security for encryption and decryption. The RSA key pairs are generated once per server startup and then shared across all threads, ensuring consistent signing and decryption behaviour.

AES encryption with Mashify functionality was also implemented. However, unlike hashing AES uses symmetric encryption causing the same key to be used for encryption and decryption, allowing plaintext messages to be encrypted and then decrypted back to its original message.

These different forms of cryptography demonstrate multiple security concepts and ensure secure communication through a distributed system.

Database Logging

Database logging was implemented using Entity Framework Core and SQL Server LocalDB. A log table was also created for recording requests made to protected endpoints to the server.

When a protected endpoint was accessed, the request route and timestamp were stored, which then allowed to keep track of the activity within the system and be monitored more effectively.

The way this was implemented was through ProtectedController which showcases how a distributed system can keep track of authenticated users, debugging and auditing.

Testing

Systems were tested both on the local and university server, this list mentions the tasks completed and problems encountered:

Tasks completed:

- All 14 development tasks completed including:
- User creation, roles and deletion
- API key authentication
- AES encryption and Mashify functionality
- RSA public key retrieval and digital signing
- Database logging and timestamps

Problems encountered:

- Client connection issues (incorrect startup) resolved via multiple startup projects
- Initial routing 404 errors fixed by adding [ApiController] and route attributes to controllers.

Client Side output:

```
C:\Users\adria\Documents\Gi x + v
...please wait...
True - User Does Exist! Did you mean to do a POST to create a new user?

What would you like to do next?
User Post NewPerson|
```

```
C:\Users\adria\Documents\Gi x + v
...please wait...
Stored

What would you like to do next?
|
```